Rapport 2 : Architecture Interne des Agents IA et Systèmes Multi-Agents

Mohamed Amine Chemkhi
Master 2 IoT – Projet *Fed-SemanticIoT*
Superviseure : Mme Nancy Awad

Novembre 2026

Ce rapport détaille les composants structurels d'un agent IA autonome basé sur un LLM local (Ollama / Llama 3.2) et définit les mécanismes de coordination multi-agents appliqués à notre projet **Fed-SemanticIoT**.

---

1 Les 4 briques formantes d'un Agent IA (LLM / Tools / History / Loop)

Un modèle de langage (LLM) interrogé seul est **stateless** (sans mémoire entre deux appels) et passif (il ne peut que prédire des tokens textuels). Pour transformer ce modèle en **Agent autonome** (selon le paradigme *ReAct* – Yao et al., 2023), notre code Python assemble quatre briques distinctes :

1.1 LLM (Le Moteur de Raisonnement)

- **Rôle :** Cerveau décisionnel de l'agent. À partir d'un contexte textuel, il analyse le problème, décide s'il a besoin d'appeler un outil externe ou formule une règle métier.
- **Implémentation dans notre code :** Instancié via le client `OpenAI(base_url="http://localhost:114 api_key="ollama")` qui route les requêtes localement vers le moteur **Ollama (`llama3.2`)**, garantissant le principe de *Zero Data Leakage*.

1.2 History (La Mémoire et gestion du Contexte)

- **Rôle :** Comme le LLM oublie tout après chaque réponse, l'historique maintient l'état de la réflexion, les instructions systèmes (`system`), les questions initiales (`user`), les réponses précédentes (`assistant`) et les retours d'outils.
- **Implémentation dans notre code :** Implémenté sous la forme d'une liste dynamique de dictionnaires Python `self.history = [{"role": "...", "content": "..."}]` enrichie à chaque itération via `self.history.append(...)`.

1.3 Tools (Les Outils / Capacités d'action)

- **Rôle :** Fonctions Python déterministes permettant à l'agent d'interagir avec son environnement local (lire un capteur IIoT, interroger une base de données locale, calculer une statistique précise).

- **Implémentation dans notre code :** Fonctions Python (ex: `tool_read_local_telemetry()`) exposées à l'agent. Lorsque le LLM génère un marqueur d'action (ex: `CALL_TOOL:Nom_Machine`), le script intercepte la demande, exécute la fonction Python et injecte le résultat dans `History`.

1.4 Loop (La Boucle de contrôle ReAct)

- **Rôle :** C'est le chef d'orchestre algorithmique (`while` ou `for`) qui fait tourner le cycle **Thought (Pensée)** $\rightarrow$ **Action (Outil)** $\rightarrow$ **Observation (Retour)** jusqu'à la résolution de la tâche.
- **Implémentation dans notre code :** La boucle d'exécution dans `run_federated_semantic_workflow` qui interroge le LLM, vérifie si une action ou une validation supplémentaire est requise, met à jour l'historique et s'arrête lorsque la règle sémantique finale est validée.

Brique Agent	Élément dans notre Code Python	Fonction principale
LLM	`client.chat.completions.create()`	Inférence locale (`llama3.2`) sans fuite réseau
History	Liste `self.history` (`system/user/assistant`)	Persistance du contexte entre les tours
Tools	`tool_read_local_telemetry(machine_id)`	Lecture des séries temporelles privées de l'usine
Loop	Boucle `for tour in range(max_iterations)`	Cycle d'exécution, appel d'outil et rétroaction

2 Concepts Clés de l'Architecture Multi-Agents

Dans notre approche **Fed-SemanticIoT**, plusieurs agents coopèrent pour extraire et partager des connaissances sans partager les données brutes des capteurs. Cette coopération repose sur quatre piliers :

2.1 Agent Specialization (Spécialisation des agents)

Plutôt que d'utiliser un agent monolithique sujet aux hallucinations sur des tâches complexes, chaque agent reçoit un rôle restreint, défini par son **System Prompt**, sa température d'inférence et ses **Tools** autorisés :

- **Agent Analyste Local (Client) :** Spécialisé dans l'analyse des logs capteurs locaux (température = 0.2) et l'extraction de règles de diagnostic anonymisées.
- **Agent Ingénieur Critique / Validateur :** Spécialisé dans la vérification physique, la détection d'hallucinations et le contrôle de confidentialité (température = 0.1, vérifie qu'aucun identifiant ou donnée brute ne fuite).
- **Agent Agrégateur Sémantique (Serveur) :** Spécialisé dans la synthèse (fusion sémantique) des règles textuelles reçues de plusieurs usines clientes.

2.2 Communication Protocols (Protocoles de communication)

Pour éviter l'ambiguïté du langage naturel libre lors du transfert de connaissances entre deux agents (Client $\leftrightarrow$ Client ou Client $\leftrightarrow$ Serveur), les échanges sont régis par un protocole structuré strict :

- **Formatage JSON standardisé :** Chaque agent doit encapsuler sa connaissance dans une structure clé-valeur stricte comprenant : `symptom`, `root_cause`, `action`, et `contains_raw_data`.
- Cela permet un parsing logiciel fiable et remplace proprement la matrice de poids de FedAvg par un objet sémantique normé.

2.3 Task Delegation (Délégation de tâches)

Face à une anomalie complexe, un agent coordinateur (ou script orchestrateur) décompose le problème en sous-tâches élémentaires et les délègue aux agents spécialisés compétents :

1. Délégation de la collecte locale et du diagnostic initial à l'*Agent Expert Local*.
2. Délégation de l'audit de sécurité et de cohérence physique à l'*Agent Validateur Critique*.
3. Délégation de la généralisation inter-sites à l'*Agent Agrégateur Serveur*.

2.4 Validation (Mécanisme de vérification et d'anti-empoisonnement)

Dans le Federated Learning classique, une mauvaise mise à jour corrompt les poids mathématiques. Dans un système fédéré sémantique, le risque majeur est l'**hallucination d'un LLM local** qui propagerait une règle de maintenance erronée ou dangereuse aux autres sites. La validation s'opère à deux niveaux :

- **Validation syntaxique et confidentialité :** Vérification par le code que l'objet JSON est valide et ne contient aucune série temporelle brute.
- **Validation sémantique croisée (*Peer-Review* par l'Agent Critique) :** Le second agent évalue la plausibilité technique de la règle. Si la règle est jugée incomplète ou risquée (`"status": "REJECTED"`), la boucle (`Loop`) renvoie les critiques au premier agent pour correction avant toute publication vers le serveur central.